



Universidade do Minho
Escola de Engenharia

Laboratórios de Informática III

Licenciatura em Engenharia Informática

Grupo 64

2022/2023

2ª fase

Alexandra Santos (A94523)
Guilherme Oliveira (A100592)
Gustavo Castro (A100482)

Braga, 3 de Fevereiro de 2023

1.- Introdução	3
2.- Estruturas de Dados	3
2.1.- Estruturas de Armazenamento	3
2.1.1.- Estrutura de User	4
2.1.2.- Estrutura de Driver	4
2.1.3.- Estrutura de Ride	4
2.1.4.- Estrutura de City	5
2.1.5.- Estrutura de Date	5
3.- Modularidade	5
3.1.- Módulos de estrutura de dados	5
3.2.- Módulos de leitura e resultado	6
3.3.- Módulos de funcionalidade	6
4.- Encapsulamento	6
4.1.- Encapsulamento nos módulos de estruturas	6
4.2.- Encapsulamento nos módulos de IO	7
4.3.- Encapsulamento nos módulos de funcionalidade	7
5.- Abstração de Dados	7
5.1.- Hashmap	8
5.2.- Listas	8
5.5.- DateMaps	8
6.- Estratégias das Queries	9
6.1.- Query 1	9
6.2.- Query 2 e 3	9
6.3.- Query 4	9
6.4.- Query 5	9
6.5.- Query 6	10
6.6.- Query 7	10
6.7.- Query 8	10
6.8.- Query 9	11
7.- Desempenho do programa	11
8.- Conclusão	12

1.- Introdução

No âmbito da unidade curricular de Laboratórios de Informática 3 (LI3), do curso de Engenharia Informática da Universidade do Minho, foi-nos solicitado a apresentação de um projeto baseado na criação de uma aplicação capaz de armazenar informações numa estrutura de dados, de forma a que essa informação possa ser usada na realização de **Queries**. Pretende-se que o programa recolha os dados da estrutura e realize os cálculos necessários à conclusão dos comandos executados.

Neste relatório é especificado o que cada módulo deste projeto faz, seja o motivo seja o objetivo. Procuramos através do mesmo, transmitir uma imagem do resultado do nosso trabalho. Diligenciamos ter boas práticas de programação, assim como implementar a modularização e o encapsulamento para que em conjunto com este documento o programa fosse de fácil compreensão e manutenção e com potencial para mudanças e adições.

2.- Estruturas de Dados

A fim de obter o melhor desempenho deste programa foi optado, em contraste com a primeira fase, por dividir os vários dados vindos dos documentos .csv em dois tipos de estruturas de dados: as de armazenamento e as de procura e organização, que serão vistas em melhor detalhe pelo capítulo de Abstração de Dados.

Depois de uma análise aprofundada, foi concluído que os antigos módulos de filtros (Filtros de Cidades e Datas) eram ineficientes e podiam ser trocados por uma estrutura de armazenamento (no caso das cidades) e uma estrutura de mapa de datas (no caso das datas).

As melhorias que estas alterações trouxeram serão, então, explicadas nas respectivas secções.

2.1.- Estruturas de Armazenamento

As estruturas de dados de armazenamento guardam toda a informação relativa aos dados principais (Utilizadores, Condutores, Viagens, Cidades e Datas) de forma a que seja facilmente acessível quando for necessária.

Desta forma, é recorrida a criação de cinco novos tipos de variáveis para: armazenar utilizadores (User); armazenar condutores (Drivers); armazenar viagens (Ride); armazenar cidades (City); e armazenar Datas (Date).

Estas estruturas não só irão armazenar a informação direta vinda da leitura do dataset, como também armazenam informações estatísticas que podemos calcular logo no parsing de dados.

Estas propriedades adicionais foram, então, criadas com intuito de permitir o acesso fácil das queries à informação calculada no parsing de dados, sendo assim possível ganhar eficiência e poupar tempo nos cálculos estatísticos, já que o programa não terá que percorrer uma segunda vez as várias estruturas armazenadas para obter informações e realizar estes cálculos.

2.1.1.- Estrutura de User

A estrutura de User armazena a informação relacionada com os Utilizadores fornecida pelos ficheiros .csv, assim como a informação do tratamento dos dados realizado no parsing de dados.

A informação fornecida pelo dataset corresponderia às propriedades **Username, Name, Gender, Car_class, Age** (calculado a partir da birth_date e da data de referência), **Account_creation, Pay_method e Account_status**.

A partir destas e da informação presentes nas Viagens são calculadas informações estatísticas guardadas nas seguintes propriedades:

- **Score:** É definido pela pontuação total acumulada dada a um Utilizador.
- **Rides:** É definido pelo número de viagens que um Utilizador realizou.
- **Money_spent:** O total gasto pelo Utilizador nas Viagens.
- **Distance:** Distância total percorrida por um Utilizador.
- **Last_ride:** Data da última viagem que um Utilizador realizou.

2.1.2.- Estrutura de Driver

A estrutura de Driver armazena a informação relacionada com os Condutores fornecida pelos ficheiros .csv, assim como a informação do tratamento dos dados realizado no parsing de dados.

A informação fornecida pelo dataset é guardada pelas propriedades **ID, Name, Age**(calculada a partir da birth_date e da data de referência), **Gender, License_plate, Account_creation e Account_status**.

À semelhança dos Utilizadores, partir destas e de informação das Viagens são calculadas informações estatísticas armazenadas nas seguintes propriedades:

- **Score:** É definido pelo array dinâmico que armazena a pontuação total e acumulada dada a um Condutor de forma Global e para cada cidade pela qual passou. A posição 0 corresponde aos valores globais.
- **Rides:** É definido pelo array dinâmico que armazena o número de viagens de um Condutor, tal como no score, guardada de forma Global e para as respectivas cidades. A posição 0 corresponde aos valores globais.
- **Money_received:** O dinheiro total que um Condutor recebeu.
- **Cities:** Define o array dinâmico que guarda as cidades por onde um Condutor passou, onde o índice da posição de cada cidade + 1 corresponde à posição onde os valores para essa cidade são armazenados nas propriedades de **Score e Rides**.
- **Counter:** O número de cidades por onde um Condutor passou.
- **Last_ride:** Guarda a data da última Viagem de um Condutor.

2.1.3.- Estrutura de Ride

A estrutura de Ride armazena apenas a informação relacionada com as Viagens fornecida pelos ficheiros .csv.

A informação fornecida será associada às propriedades: **ID, Date, Driver, User, City, Distance, Score_user, Score_driver, Tip e Comment**.

Esta estrutura não tem parâmetros com informação de tratamento de dados, pois, no contexto do projeto, nenhuma informação adicional seria útil para as várias queries e os vários outputs.

Em contraste à estrutura de Ride da primeira fase, as propriedades **Driver** e **User** já não armazenam o **Id** e o **Username**, respetivamente. Chegou-se à conclusão de que seria mais útil e eficiente armazenar, em vez, os apontadores para os respectivos, pois aquando da manipulação de Viagens o acesso a ambos será mais direto assim, em contraste a ter de procurar ou o **Id** ou o **Username** nos respectivos **Hashmaps**.

2.1.4.- Estrutura de City

A estrutura de cidade é definida pela informação relativa à cidade, armazenando as estatísticas globais da cidade, assim como uma lista de Condutores que passaram por essa cidade.

Esta lista objetiva não só listar todos os Condutores de uma cidade, como também ordená-los pelas exigências da query 7. Inicialmente a lista iria começar desordenada, no momento de parsing, mas seria ordenada após a primeira ocorrência da query 7, mantendo-se ordenada durante o resto do programa.

A sua estrutura apresenta as seguintes definições:

- **City:** Armazena o nome da Cidade.
- **money_spent:** Dinheiro total gasto numa Cidade.
- **rides:** Número de viagens realizadas numa Cidade.
- **drivers:** Lista de drivers que fizeram viagens numa Cidade.

2.1.5.- Estrutura de Date

A estrutura de Date armazena uma Data, visando a aproveitar a localidade espacial apresentada num array para tal.

Será então definida por uma array de 3 shorts, onde a posição 0 será o dia, a posição 1 será o mês e a posição 2 o ano.

3.- Modularidade

Tendo em consideração a melhor organização e desempenho do programa, as várias funcionalidades e estruturas foram divididas por diversos módulos.

Estes módulos foram divididos por quatro tipos:

- Módulos de leitura e resultado (io);
- Módulos de estruturas de dados (structs);
- Módulos de queries (queries);
- Módulos de funcionalidade (src);

3.1.- Módulos de estrutura de dados

Este tipo de módulos tem como objetivo definir as estruturas de dados usadas ao longo do programa, assim como funções úteis à sua utilização (criação, destruição, busca das estruturas, comparações, impressões, etc...). Para esta tipologia, foram criados os seguintes módulos:

- **Date** (Armazenamento de uma data);

- **User** (Armazenamento de utilizadores);
- **Driver** (Armazenamento de condutores);
- **Ride** (Armazenamento de viagens);
- **City** (Armazenamento de cidades);
- **List** (Armazenamento de vários Dados, tipicamente ordenados);
- **DateMap** (Armazenamento de vários Dados com filtro por datas);
- **Hashmap** (Armazenamento de vários Dados com filtro por chaves);
- **Global** (Armazenamento global).

3.2.- Módulos de leitura e resultado

Nesta tipologia, o objetivo é definir os métodos de leitura do input do utilizador e de criação de resultados dependentes do que foi lido no mesmo input. Assim sendo, recorreu-se à utilização dos quatro seguintes módulos:

- **Parser** (módulo do modo batch/leitura dos datasets);
- **Interactive** (módulo do modo interativo);
- **Interpreter** (módulo de interpretação de Inputs);
- **Page** (módulo de paginação de resultados do modo interativo).

3.3.- Módulos de funcionalidade

Nos módulos de funcionalidade, o objetivo é tratar a informação recebida, chamar funções dos dois módulos anteriores, e devolver resultados baseados no que é recebido. Desta forma, são definidos os seguintes módulos:

- **Main** (Chama o programa);
- **Utils** (Utilidades, conversões, etc...);
- **Test** (Execução de testes).

4.- Encapsulamento

De forma a encapsular o programa, às várias funcionalidades de cada módulo são atribuídas permissões. Estas permissões irão definir quais módulos têm acesso a quais funcionalidades, permitindo, desta forma, uma maior segurança no programa, sendo que algumas funcionalidades podem até apenas poder ser acedidas pelo próprio módulo (no caso de funções auxiliares a funções principais).

4.1.- Encapsulamento nos módulos de estruturas

Todos os módulos de estruturas de dados são independentes em relação aos módulos exteriores, recorrendo excepcionalmente a funções do módulo **Utils** para simplificar os seus processos.

Tirando esse acesso ao exterior, comunicam-se apenas entre si, incorporando-se um nos outros quando necessário.

O módulo mais independente são os módulos de **Date**, **Hashmap** e **List**, que apenas precisam deles próprios para funcionar.

Os restantes módulos “montam-se” no topo destes, sendo que ambos o **User** e o **Driver** tem acesso ao **Date** por armazenarem Datas, e ambos o **City** e o **DateMap** tem acesso ao **List** e ao **Date** (no caso do **DateMap**), por o usarem para armazenar listas de Condutores e lista de Viagens, respetivamente.

Já o módulo **Ride** não só acede ao **Date** (para guardar a data da Viagem), como também ao **User** e **Driver**, por armazenar os respectivos apontadores.

O módulo **Global** tem, porém, acesso a todos estes módulos (por guardar toda a informação).

4.2.- Encapsulamento nos módulos de IO

Em contraste à categoria anterior, os quatro módulos de IO divergem bastante em termos de permissões de acesso aos restantes módulos.

O módulo **Interactive** tem acesso ao módulo **Interpreter**, de forma a chamar as funções de interpretação presentes no mesmo; e ao módulo **Global**, conseguindo, assim, manter a mesma Global ao longo da interpretação da informação do input do utilizador.

O módulo **Interpreter** tem acesso aos módulos de estruturas, para poder criar todas as estruturas necessárias consoante a interpretação do input do utilizador, ao módulo **Utils**, visto que são necessárias funções deste módulo para conversões, criação e comparações de dados.

O módulo **Page** tem acesso ao módulo da estrutura de dados **List** de modo a criar páginas de outputs de lista que se pretende imprimir, e ao módulo **Utils**, uma vez que são necessárias funções deste módulo.

Por fim, o módulo **Parser** tem acesso ao módulo **Interpreter**, de forma a chamar as funções de interpretação, e ao módulo **Global**, de modo a conseguir manter a mesma Global ao longo da interpretação da informação do input do utilizador.

4.3.- Encapsulamento nos módulos de funcionalidade

Os módulos de funcionalidade, tal como os da categoria anterior, divergem, também, em permissões, sendo que o módulo **Utils** apenas tem acesso a ele próprio.

O módulo **Test** apenas tem acesso ao módulo **Utils** e ao módulo **Parser**, para as comparações, conversões e criação de dados para a execução dos testes.

Por último, o módulo **Main** do programa, que o executa, tem acesso ao módulo **Parser**, usando a sua função para iniciar a leitura e interpretação do input e ao módulo **Global**, conseguindo, assim, criar a estrutura global que sustentará a execução inteira do programa.

5.- Abstração de Dados

A fim de obter a melhor organização e desempenho possível, foi preferido o uso de **Hashmaps**, **Lists** e **DateMaps** para armazenamento dos dados discutidos anteriormente.

5.1.- Hashmap

Os hashmaps foram alterados para um sistema de **open-addressing** dinâmico, em contraste ao antigo sistema de chaining fixo, pois, depois de uma análise à eficiência, achamos que este sistema iria encaixar-se melhor no nosso projeto.

Para melhor a eficiência, escolhemos usar quadric probing pois este lida melhor com as colisões, fazendo, de forma geral, o sistema de procura melhor e mais eficiente. Assim sendo, cada posição do hashmap será constituída por um **HashmapNode**, que armazena o apontador para a chave que define o elemento, o apontador para o próprio elemento, e o char status que poderá ser 0 ou 1, que está explicado mais à frente. A lógica de hash será a mesma, é criada uma chave hash que irá definir a posição em que o elemento será armazenado, sendo que em caso de colisão esta posição saltará em quadrados ($+1^2$, $+2^2$, $+3^2$, etc...) até encontrar uma posição livre para inserir o elemento, ou concluir que é necessário aumentar o hashmap, isto se voltar à posição inicial.

O hashmap terá sempre um tamanho primo, para reduzir o número de colisões, sendo que será sempre inicializado com 11 elementos. Sempre que for redimensionado passará a ter o número primo mais próximo ao dobro do tamanho. Aquando do redimensionamento do **Hashmap**, todos os seus elementos passarão ao status 0, e serão percorridos para serem reposicionados, retornando apenas ao status 1 quando uma nova posição adequada para estes for encontrada.

Por fim, foi ainda criada uma nova função map que aplica uma função x a todos os elementos de um hashmap. Esta função foi criada a pensar na query 2 e 3, mas poderá ser utilizada para outros propósitos.

5.2.- Listas

Inicialmente foram criadas listas ligadas mas optamos antes por utilizar arrays dinâmicos, mas achamos importante explicar na mesma qual foi o nosso raciocínio

A criação destas listas utiliza-se em ordenações e armazenamento de informação - como ordenação de drivers e users para as queries 2 e 3. Para isso foram incluídas funções para funcionar com ambos os modos. A criação de listas era igual em ambos os modos: usando a createList, que é responsável por inserir um elemento à lista dando realloc à lista existente para que este tenha espaço. Foram criadas várias funções para manusear estas listas como addList que adiciona um elemento a lista, swap que troca dois elementos da lista e muitas mais. Inicialmente são criadas as listas de drivers, users e rides, na glob, que depois serão ordenadas tendo em conta a necessidade de cada query. O facto de as queries usarem arrays dinâmicos faz com que usufruam de localidade espacial e depois de realizada uma query que ordene a lista a segunda vez que seja chamada terá substancialmente menos tempo de execução uma vez que a ordenação é o processo mais demorado da query.

5.5.- DateMaps

Depois de uma melhor análise à organização e usos das datas, decidimos que seria melhor converter os DateFilters num novo tipo: os DateMaps.

Os DateMaps armazenam um ano e um array que pode ser visto como os meses e os dias desses meses ([MÊS-1][DIA-1]). Esse array irá guardar dentro de cada posição uma lista ligada de apontadores para as viagens que aconteceram nessas datas.

Assim sendo, como a maior parte dos usos estatísticos das viagens são filtrados por data, este mapa faz com que as viagens sejam mais facilmente acessíveis, sendo que apenas é preciso ir a uma posição do array para encontrar uma lista de viagens que se realizaram num certo dia.

6.- Estratégias das Queries

6.1.- Query 1

Para a realização desta query o raciocínio foi simples, mas eficaz. A função foi feita de modo a que verificasse em que hashmap deveria procurar, no de users ou de drivers, e a que, após a seleção do tipo de utilizador, usasse as funções feitas para o fetch de informação de hashmaps para adquirir a informação necessária a partir do ID fornecido.

6.2.- Query 2 e 3

As estratégias da query 2 e 3 são semelhantes.

A fim de realizar a query 2 e 3, aproveitou-se o momento de parsing para criar uma **List** com todos os Drivers e uma **List** com todos os **Users**.

Tendo essas **List** criadas, a query 2 e 3 limitam-se a ordenar as listas usando as funções para tal (**sortList** em junção com a **driverCmp** ou **userCmp**), e dar print aos primeiros N elementos.

Numa segunda, terceira, etc... execução desta query, a query 2 e 3 limitam-se a imprimir os N resultados, visto que as **List** agora estão sempre ordenadas.

6.3.- Query 4

No início do projeto, quando hashmap de City ainda continha cada viagem associada a uma cidade a cada elemento, a query 4 percorria esse hashmap e as viagens referentes a cidade em questão ela acumulava o preço da viagem numa variável de preço total e incrementava uma variável de contagem declarada inicialmente a 0. Após este processo calculava então o preço médio.

Com a reestruturação, toda a informação necessária a query 4 encontra-se na nova estrutura City, pois todos os cálculos estatísticos foram feitos no parsing. Os parâmetros money_spent e rides terão os mesmos valores que as variáveis anteriormente referidas, mas com substancialmente menos tempo necessário para as adquirir.

6.4.- Query 5

Inicialmente para a query 5 foi criada uma função auxiliar que percorria a antiga estrutura de datas que continha todas as ocorrências de todas as datas na base de dados associadas ao respetivo user, driver ou ride e retirava todas aquelas que estavam dentro das datas pretendidas e eram referentes a datas de viagens para uma lista ligada auxiliar. De seguida, a query 5 recebia essa lista ligada e percorria-a calculando o preço médio pretendido.

Com a nova estrutura de dateMap que organiza as viagens por datas, a query 5 apenas tem que percorrer na hashmap de dateMap as listas ligadas de todos os dias entre a data A e a data B somando o preço total com a ajuda de funções de ordem superior e contando o número de viagens realizadas. Por fim, só resta calcular o preço médio.

6.5.- Query 6

Durante o processo de resolução das queries a query 6 sofreu algumas alterações. No início, quando hashmap de City ainda continha cada viagens associada a uma cidade a cada elemento, a query 6 era feita percorrendo todas as estruturas city e as uma ocorrência fosse da cidade em questão os valores da viagem necessários eram adicionados a variáveis de estatística que no final contabilizavam a distância média percorrida

Com a reestruturação, todos os dados estatísticos necessários a query passaram a ser feitos no parsing, ou seja, no hashmap de City cada cidade tem os valores pretendidos fazendo com que a query 6 se baseasse a adquirir a City da cidade pretendida e a retirar dela a informação já tratada.

6.6.- Query 7

Inicialmente a query 7 usava a lista ligada de viagens na estrutura da cidade dada como argumento para rapidamente adicionar ordenadamente por avaliação média a uma lista ligada auxiliar que teria os N primeiros condutores pela avaliação média. Infelizmente com a reestruturação da estrutura de dados City não foi possível usar mais a lista ligada, portanto foi usada uma estratégia baseada nas funções de ordem superior desenvolvidas para as novas listas. Assim a query 7 passa a verificar se a cidade dada como argumento existe e, caso exista, usa estas funções para percorrer a lista de rides ordenando-a da forma pretendida e depois imprime os N primeiros condutores no ficheiro de output.

6.7.- Query 8

Durante o processo de resolução das queries a query 8 sofreu algumas alterações. Inicialmente, a query extraia a informação das viagens, condutores e utilizadores do hashmap e colocava numa lista. De seguida, ordenava essa lista conforme o enunciado - com base na antiguidade do perfil do condutor, na antiguidade do perfil do utilizador e por ordem crescente do id da viagem. Por fim, seriam impressos os resultados com os campos pretendidos.

De momento, a query 8 tem como objetivo listar todas as viagens nas quais o utilizado e o condutor sejam de um determinado género e possuam perfis com X ou mais anos.

O output é ordenado de forma a que as contas mais antigas apareçam primeiro, mais especificamente, ordenadas pela conta mais antiga de condutor e, se necessário, pela conta do utilizador. Caso persistam os empates, ordena-se através do id da viagem por ordem crescente.

6.8.- Query 9

Ao longo do projeto a query 9 sofreu várias alterações de raciocínio. Inicialmente, a query 9 iria percorrer o hashmap de viagens e em cada elemento se verificasse que a viagem satisfazia os parâmetros estipulados seria adicionada a uma lista ligada. Essa lista depois seria impressa como output.

Com a nova reestruturação das estruturas de dados as viagens passaram a estar organizadas por datas o que facilitou em muito a aquisição das viagens necessárias. Através das funções de ordem superior foi possível percorrer todas as viagens entre uma data e outra e fazer a seleção das que satisfazem os parâmetros para que sejam adicionadas à lista que posteriormente será impressa como output.

7.- Desempenho do programa

Considerando toda a estrutura e funcionamento descritos até agora, foram feitas medições que avaliam o desempenho do programa.

Para fazer essas medições, foi criado um programa de testes que recebendo um teste (uma pasta contendo um input.txt com comandos e uma pasta com os resultados desses comandos) avalia o desempenho do programa principal.

Os resultados desse programa foram registados nas tabelas seguintes:

Sistema 1 - Lenovo IdeaPad Flex 5 - AMD Ryzen 7 5700U, 1801MHz, 8 Nucleos, 16 processadores Lógicos - Ubuntu 22.04.01 LTS

Processos	data-regular	data-regular-err	data-large	data-large-err
Parse	2.043568 s	2.032045 s	39.790712 s	38.231810 s
Query 1	0.000001 s	0.000000 s	0.000000 s	0.000000 s
Query 2	0.003032 s	0.003197 s	0.138956 s	0.125031 s
Query 3	0.040873 s	0.039588 s	0.718075 s	0.699426 s
Query 4	0.000004 s	0.000003 s	0.000000 s	0.000000 s
Query 5	0.015670 s	0.000010 s	0.000000 s	0.000000 s
Query 6	0.000036 s	0.000009 s	0.000615 s	0.000596 s
Query 7	0.025496 s	0.022768 s	0.735295 s	0.720852 s
Query 8	0.888786 s	0.829215 s	16.556570 s	15.950864 s
Query 9	0.013516 s	0.000008 s	0.000006 s	0.000005 s
Total	3.141377 s	3.054552 s	72.464212 s	70.098250 s
Memory	248	251	2549	2487
Leaks	0.0	0.0	0.0	0.0

Sistema 2 - Lenovo ThinkPad - Core i7 + 16GB Ram + Ubuntu 22.04.01 LTS

Processos	data-regular	data-regular-err	data-large	data-large-err
Parse	1.684969 s	1.635504 s	31.179533 s	30.465048 s
Query 1	0.000002 s	0.000001 s	0.000001 s	0.000001 s
Query 2	F - 0.002589 s S - 0.000001 s	F - 0.002491 s S - 0.000008 s	F - 0.086331 s S - 0.000178 s	F - 0.078528 s S - 0.000185 s
Query 3	F - 0.033388 s S - 0.000001 s	F - 0.032836 s S - 0.000019 s	F - 0.642574 s S - 0.000001 s	F - 0.609372 s S - 0.000001 s
Query 4	0.000006 s	0.000003 s	0.000001 s	0.000001 s
Query 5	0.008388 s	0.000008 s	0.000001 s	0.000001 s
Query 6	0.000050 s	0.000005 s	0.000445 s	0.000486 s
Query 7	F - 0.017372 s S - 0.000001 s	F - 0.015184 s S - 0.000020 s	F - 0.643354 s S - 0.000001 s	F - 0.582147 s S - 0.000001 s
Query 8	F - 0.642231 s S - 0.021986 s	F - 0.568160 s S - 0.013902 s	F - 12.774711 s S - 0.134294 s	F - 12.317197 s S - 0.123433 s
Query 9	0.013772 s	0.000007 s	0.000006 s	0.000006 s
Total	2.47787 s	2.348702 s	55.991589 s	54.898613 s
Memory	264	255	2602	2514
Leaks	0.0	0.0	0.0	0.0

8.- Conclusão

Nesta fase final do projeto conseguimos evoluir o nosso conhecimento sobre os princípios fundamentais da Engenharia de Software, designadamente modularidade, reutilização, encapsulamento e abstração de dados, assim como aperfeiçoar a nossa capacidade de programação e trabalho em equipa.

Foi possível compreender a estratégia e a implementação demonstrável na leitura e interpretação dos dados de entrada e na utilização do modo de operação interativo. Outro objetivo atingido, foi a criação de todas as queries com base no catálogo de dados dos três ficheiros de entrada.